PAGE:
DATE: / /

Task - 10 TechNeeKX

Q- Learn commands of Django to start a project.

Step 1: Create a Project Folder

```
mkdir myproject
cd myproject
```

Step 2: Create a virtual Environment

```
python -m venv venv
```

This creates a folder named venv containing virtual environment.

Step 3: Activate the Virtual Environment

```
.venv\Scripts\activate
```

After activation, your terminal looks like
(venv) C:\Users\YourName\myproject>

Step 4: Install Django

```
pip install django
```

check installation:

```
django-admin --version
```

PAGE:
DATE: / /

Step 5: Create a Django Project

django-admin startproject mysite

A folder named mysite will be created

Step 6: Go inside the Project

cd mysite

Step 7: Run the Development Server

python manage.py runserver

Output:
Starting development server at http://127.0.0.1:8000/

Open browser and visit:
http://127.0.0.1:8000
We will see Django Welcome Page

Step 8: Stop the Server
Ctrl + C

Step 9: Create an App

python manage.py startapp blog

PAGE:
DATE: / /

Step 10: Apply Database Migrations

python manage.py migrate

Step 11: Create an Admin User

Python manage.py createsuperuser

Enter Username, Email, password

Run → python manage.py runserver

Open :

http://127.0.0.1:8000/admin

then Login.

Common Django Commands:

1) django-admin startproject projectname
Creates new Django Project

2) python manage.py runserver
Start development server

3) python manage.py startapp appname
Create new app

4) python manage.py makemigrations
Create migration files

PAGE:
DATE: / /

5) python manage.py migrate
   Apply migrations

6) Python manage.py createsuperuser
   Create admin user

7) python manage.py shell
   Open Django shell

8) python manage.py collectstatic
   Collect static files

9) python manage.py test
   Run tests

Q2- What is a Virtual Environment?

It is an isolated python environment that keeps the packages of one project separate from other Python projects on the same computer.

Without venv:

Computer:
- Django 4.2
- Flask
- Numpy
- Pandas

With env:

Computer

```
├── Project A
│   └── env
│       └── Django 5.0
│
└── Project B
    └── env
        └── Django 4.2
```

Advantages:

No Package conflicts
Easy to manage dependencies, makes projects portable.

3- Structure of Django framework

After creating Django Project

The folder structure is:

```
mysite/
├── manage.py
└── mysite/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

PAGE:
DATE: / /

With env:

Computer
├ Project A
: └ env
: Django 5.0
:
:— Project B
: └ env
: Django 4.2

Advantages:

- No Package conflicts
- Easy to manage dependencies, makes projects portable.

Q3- Structure of Django framework

After creating Django Project

The folder structure is:

```
mysite/
:— manage.py
:— mysite/
   :— init_.py
   — settings.py
   :— urls.py
   — asgi.py
   :— wsgi.py
```

PAGE:
DATE: / /

If you create an app:

Python manage.py startapp blog

The structure becomes:

```
mysite/
├── manage.py
├── blog/
│   ├── migrations/
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   ├── views.py
│   └── init.py
│
└── mysite/
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Explaination of Imp files

1. manage.py (control center of Project)
   Used to execute Django commands
   Ex:

   Python manage.py runserver
   Python manage.py migrate
   Python manage.py createsuperuser

2) settings

Contains
as:
. Install
. Datab
. Middle
. Templ
. Stati
. Secur
. Time
. Lang

Ex:

3) Wor
M

Ex

PAGE:
DATE: / /

2) settings.py

Contains all project configurations such as:

- Installed apps
- Database settings
- Middleware
- Templates
- Static Files
- Security settings
- Time zone
- Language

Ex:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
]
```

3) urls.py

MAPS urls to views

Ex:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home),
]
```

PAGE:
DATE: / /

4) views.py

Contains the business logic, It processes requests and return responses.

Ex:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello World")
```

5) models.py

Defines database tables using Python classes.

Ex:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Django converts this model into a database table through migrations.

6) admin.py

PAGE:
DATE: / /

Registers models so they appear in the Django Admin Panel.

Ex:

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

2) apps.py

Stores configuration information for the app.

Ex:

```
from django.apps import AppConfig

class BlogConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'blog'
```

3) tests.py

Contains automated tests for your application.

PAGE:
DATE: / /

Ex:

```
from django.test import TestCase

class TestStudent(TestCase):
    pass
```

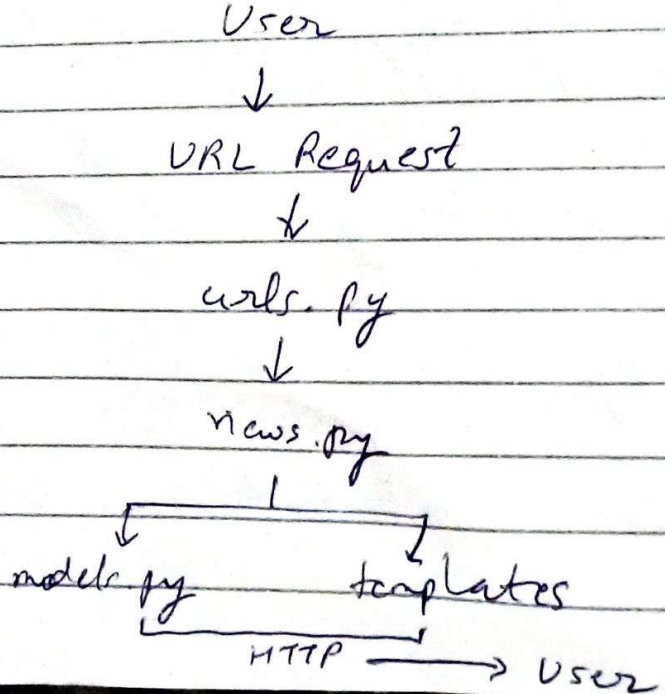
9) migrations/

Stores migration files that track database changes

Ex:

```
python manage.py makemigrations
python manage.py migrate
```

Django Architecture (MVT)



PAGE:
DATE: / /

Model: Manages data and database
View: Contains app logic and handle request
Template: Defines the HTML pages shown to user.